Monitor 서비스 구조 분석

개요

Monitor 서비스는 Omni 네트워크의 전체적인 상태를 모니터링하고 메트릭을 수집하는 핵심 컴포넌트입니다. 개별 애플리케이션에서 측정되지 않는 네트워크 전반의 메트릭을 추적하며, 스마트 컨트랙트, 검증자 상태, 계정 잔액, 크로스체인 메시지 등을 모니터링합니다.

전체 아키텍처

서비스 구조

```
Monitor Service
├── app/ # 핵심 애플리케이션 로직
| ├── app.go # 메인 애플리케이션 진입점
| ├── config.go # 설정 관리
| └── metrics.go # 기본 메트릭 정의
├── cmd/ # CLI 명령어 인터페이스
├── account/ # 계정 잔액 모니터링
├── validator/ # 검증자 상태 추적
├── xmonitor/ # 크로스체인 모니터링
├── contract/ # 스마트 컨트랙트 모니터링
├── staking/ # 스테이킹 모니터링
├── supply/ # 토큰 공급량 모니터링
```

```
├── loadgen/ # 로드 생성 및 테스트
├── cctpgen/ # CCTP 메시지 생성
├── xfeemngr/ # 크로스체인 수수료 관리
└── routerecon/ # 라우트 정찰 및 분석
```

핵심 컴포넌트 분석

1. 메인 애플리케이션 (app/app.go)

Monitor 서비스의 핵심 진입점으로, 모든 모니터링 컴포넌트를 초기화하고 관리합니다.

주요 함수: `Run`

```go
func Run(ctx context.Context, cfg Config) error {
log.Info(ctx, "Starting monitor service")
buildinfo.Instrument(ctx)
// 모니터링 서버 시작 (메트릭 엔드포인트)
monitorChan := serveMonitoring(cfg.MonitoringAddr)
// 네트워크 설정 및 클라이언트 초기화
portalReg, err := makePortalRegistry(ctx, cfg.Network, cfg.RPCEndpoints)
network, err := netconf.AwaitOnExecutionChain(ctx, cfg.Network, portalReg, ...)
ethClients, err := initializeEthClients(ctx, network.EVMChains(), ...)
// 각 모니터링 컴포넌트 시작
if err := account.StartMonitoring(ctx, solvNetwork, solvEthClients); err != ni
return errors.Wrap(err, "start account monitor")
```

```go
}
if err := contract.StartMonitoring(ctx, network, cfg.RPCEndpoints, ethClients);
return errors.Wrap(err, "monitor contracts")
}
if err := xmonitor.Start(ctx, network, xprov, cprov, ethClients); err != nil {
return errors.Wrap(err, "start xchain monitor")
}
// 백그라운드 모니터링 고루틴들 시작
go runHistoricalBaselineForever(ctx, network, cprov)
go monitorUpgradesForever(ctx, cprov)
go validator.MonitorForever(ctx, cprov)
go staking.MonitorForever(ctx, cprov)
go supply.MonitorForever(ctx, cprov, network, ethClients)
// 서비스 실행 대기
select {
case <-ctx.Done():
return nil
case err := <-monitorChan:
return err
}
}
```

초기화 과정

1. **모니터링 서버 시작**: HTTP 서버로 `/metrics` 엔드포인트 제공
2. **네트워크 설정**: Portal Registry를 통한 네트워크 구성 로드
3. **RPC 클라이언트 초기화**: 각 체인별 Ethereum 클라이언트 설정
4. **Provider 생성**: 크로스체인(`xprovider`)과 합의체인(`cprov`) Provider 생성
5. **컴포넌트 시작**: 각 모니터링 모듈을 순차적으로 시작

2. 설정 관리 (app/config.go)

Monitor 서비스의 모든 설정을 관리하는 구조체입니다.

Config 구조체

```
type Config struct {
RPCEndpoints xchain.RPCEndpoints // 체인별 RPC 엔드포인트
Network netconf.ID // 네트워크 식별자
MonitoringAddr  string  // 모니터링 서버 주소 (:26660)
PrivateKey  string  // 개인키 파일 경로
FlowGenKey  string  // FlowGen용 개인키
HaloCometURL  string  // Halo CometBFT URL
SolverAddress  string  // Solver 서비스 주소
HaloGRPCURL  string  // Halo gRPC URL (선택사항)
LoadGen loadgen.Config // 로드 생성 설정
XFeeMngr xfeemngr.Config // 수수료 관리 설정
DBDir  string  // 데이터베이스 디렉토리
RouteScanAPIKey  string  // RouteScan API 키
```

```
}
```

기본 설정

```
func  DefaultConfig() Config {
return Config{
PrivateKey: "monitor.key",
MonitoringAddr: ":26660",
DBDir: "./db",
}
}
```

3. 메트릭 시스템 (app/metrics.go)

Monitor 서비스 자체의 기본 메트릭을 정의합니다.

주요 메트릭

```
var (
// RPC 동기화 차이 모니터링
syncDiff = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "rpc",
Name: "sync_diff",
Help: "Maximum sync difference (concurrent latest heights) per chain",
```

```
}, []string{"chain"})

// 히스토리컬 베이스라인 성능

histBaseline = promauto.NewHistogramVec(prometheus.HistogramOpts{

Namespace: "monitor",

Subsystem: "cprovider",

Name: "historical_baseline_seconds",

Help: "Baseline time (in seconds) to stream historical approved attestation",

Buckets: prometheus.ExponentialBucketsRange(time.Second.Seconds(), time.Hour.Sec

}, []string{"chain", "size"})

// 업그레이드 추적

plannedUpgradeGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Name: "planned_upgrade",

Help: "Height of current planned (non-processed) upgrade by name",

}, []string{"upgrade"})

appliedUpgradeGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Name: "applied_upgrade",

Help: "Height of last applied (processed) upgrade by name",

}, []string{"upgrade"})

)
```

4. CLI 인터페이스 (cmd/)

Monitor 서비스의 명령줄 인터페이스를 제공합니다.

명령어 구조 (cmd/cmd.go)

```
func New() *cobra.Command {
cmd := libcmd.NewRootCmd(
"monitor",
"Service that monitors Omni network metrics not measured in individual apps (e.g
)
cfg := monitor.DefaultConfig()
bindRunFlags(cmd.Flags(), &cfg)
bindLoadGenFlags(cmd.Flags(), &cfg.LoadGen)
bindXFeeMngrFlags(cmd.Flags(), &cfg.XFeeMngr)
logCfg := log.DefaultConfig()
log.BindFlags(cmd.Flags(), &logCfg)
cmd.RunE = func(cmd *cobra.Command, _ []string) error {
ctx, err := log.Init(cmd.Context(), logCfg)
if err != nil {
return err
}
return monitor.Run(ctx, cfg)
}
return cmd
```

```
}
```

플래그 바인딩 (cmd/flags.go)

주요 설정 플래그들:

```
func bindRunFlags(flags *pflag.FlagSet, cfg *monitor.Config) {
netconf.BindFlag(flags, &cfg.Network)
xchain.BindFlags(flags, &cfg.RPCEndpoints)
flags.StringVar(&cfg.PrivateKey, "private-key", cfg.PrivateKey, "The path to the
flags.StringVar(&cfg.MonitoringAddr, "monitoring-addr", cfg.MonitoringAddr, "The
flags.StringVar(&cfg.HaloCometURL, "halo-url", cfg.HaloCometURL, "The URL of the
flags.StringVar(&cfg.HaloGRPCURL, "halo-grpc-url", cfg.HaloGRPCURL, "The gRPC UR
flags.StringVar(&cfg.DBDir, "db-dir", cfg.DBDir, "The path to the database direc
}
```

모니터링 서버

HTTP 서버 구성

Monitor 서비스는 내장 HTTP 서버를 통해 메트릭과 디버깅 정보를 제공합니다.

```
func serveMonitoring(address string) <-chan error {
errChan := make(chan error)
go func() {
```

```go
mux := http.NewServeMux()

mux.Handle("/metrics", promhttp.Handler()) // Prometheus 메트릭

// pprof 디버깅 엔드포인트

mux.HandleFunc("/debug/pprof/", pprof.Index)

mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)

mux.HandleFunc("/debug/pprof/profile", pprof.Profile)

mux.HandleFunc("/debug/pprof/symbol", pprof.Symbol)

mux.HandleFunc("/debug/pprof/trace", pprof.Trace)

srv := &http.Server{

Addr: address,

ReadHeaderTimeout: 5 * time.Second,

IdleTimeout: 5 * time.Second,

WriteTimeout: 5 * time.Second,

Handler: mux,

}

errChan <- errors.Wrap(srv.ListenAndServe(), "serve monitoring")

}()

return  errChan

}
```

제공되는 엔드포인트

- `/metrics` : Prometheus 형식의 메트릭 데이터
- `/debug/pprof/` : Go 프로파일링 정보

- `/debug/pprof/profile` : CPU 프로파일
- `/debug/pprof/trace` : 실행 추적 정보

Provider 시스템

CChain Provider

합의 체인과의 통신을 위한 Provider입니다.

```
func newCProvider(ctx context.Context, cfg Config) (cchain.Provider, error) {
if cfg.HaloGRPCURL != "" {
log.Debug(ctx, "Using grpc cprovider", "url", cfg.HaloGRPCURL)
encConf, err := haloapp.ClientEncodingConfig(ctx, cfg.Network)
if err != nil {
return nil, errors.Wrap(err, "client encoding config")
}
return cprovider.NewGRPC(cfg.HaloGRPCURL, cfg.Network, encConf.InterfaceRegistr
}
log.Debug(ctx, "Using comet cprovider", "url", cfg.HaloCometURL)
c, err := comethttp.New("tcp://"+cfg.HaloCometURL, "/websocket")
if err != nil {
return nil, errors.Wrap(err, "new tendermint client")
}
return cprovider.NewABCI(c, cfg.Network), nil
```

```
}
```

Provider 선택 로직:

- gRPC URL이 설정된 경우: gRPC Provider 사용 (더 빠름)
- 그렇지 않은 경우: ABCI Provider 사용 (CometBFT WebSocket)

XChain Provider

크로스체인 통신을 위한 Provider입니다.

```
xprov := xprovider.New(network, ethClients, cprov)
```

데이터베이스 시스템

인덱서 데이터베이스

크로스체인 이벤트를 인덱싱하기 위한 데이터베이스입니다.

```
func startIndexer(
ctx context.Context,
cfg Config,
network netconf.Network,
xprov xchain.Provider,
) error {
var db dbm.DB
if cfg.DBDir == "" {
```

```
log.Warn(ctx, "No --db-dir provided, using in-memory DB", nil)

db = dbm.NewMemDB()

} else {

var err error

db, err = dbm.NewGoLevelDB("indexer", cfg.DBDir, nil)

if err != nil {

return errors.Wrap(err, "new golevel db")

}

}

return indexer.Start(ctx, network, xprov, db)

}
```

데이터베이스 옵션:

- **LevelDB**: 프로덕션 환경에서 사용 (`--db-dir` 지정)
- **MemDB**: 테스트 환경에서 사용 (메모리 내 저장)

모니터링 컴포넌트 통합

Monitor 서비스는 다음과 같은 순서로 각 모니터링 컴포넌트를 시작합니다:

1. **Account Monitor**: 계정 잔액 및 nonce 추적
2. **Contract Monitor**: 스마트 컨트랙트 상태 모니터링
3. **XChain Monitor**: 크로스체인 메시지 및 오프셋 추적
4. **Indexer**: 크로스체인 이벤트 인덱싱
5. **XFee Manager**: 크로스체인 수수료 관리

6. **CCTP Generator**: CCTP 메시지 생성 및 테스트

백그라운드 고루틴

다음 모니터링 작업들이 백그라운드에서 지속적으로 실행됩니다:

```
go runHistoricalBaselineForever(ctx, network, cprov) // 히스토리컬 성능 베이스라인
go monitorUpgradesForever(ctx, cprov) // 업그레이드 추적
go routerecon.ReconForever(ctx, network, cfg.RouteScanAPIKey, xprov, ethClients
go validator.MonitorForever(ctx, cprov) // 검증자 상태 모니터링
go monitorPublicRPCForever(ctx, network, ethClients) // 퍼블릭 RPC 동기화 모니터링
go monitorOmniEVMGasTipForever(ctx, network, ethClients) // 가스 팁 모니터링
go staking.MonitorForever(ctx, cprov) // 스테이킹 모니터링
go supply.MonitorForever(ctx, cprov, network, ethClients) // 토큰 공급량 모니터링
```

에러 처리 및 복구

Monitor 서비스는 각 컴포넌트의 실패를 독립적으로 처리합니다:

- **치명적 오류**: 서비스 전체 종료 (네트워크 설정 실패 등)
- **비치명적 오류**: 로그 기록 후 계속 실행 (개별 모니터링 실패)
- **재시도 로직**: 각 모니터링 컴포넌트는 자체적인 재시도 메커니즘 보유

성능 최적화

연결 관리

```go
func initializeEthClients(ctx context.Context, chains []netconf.Chain, endpoint

rpcClientPerChain := make(map[uint64]ethclient.Client)

for _, chain := range chains {

// ... 클라이언트 생성 ...

go c.CloseIdleConnectionsForever(ctx) // 유휴 연결 정리

rpcClientPerChain[chain.ID] = c

}

return rpcClientPerChain, nil

}
```

메모리 관리

- **ResetGaugeVec**: 동적 라벨을 가진 메트릭의 메모리 누수 방지
- **고루틴 관리**: Context를 통한 우아한 종료
- **연결 풀링**: 각 체인별 독립적인 RPC 클라이언트 관리

Monitor 서비스는 Omni 네트워크의 전체적인 건강성을 보장하는 핵심 인프라로, 모든 컴포넌트의 상태를 실시간으로 추적하고 문제 발생 시 즉시 알림을 제공합니다.